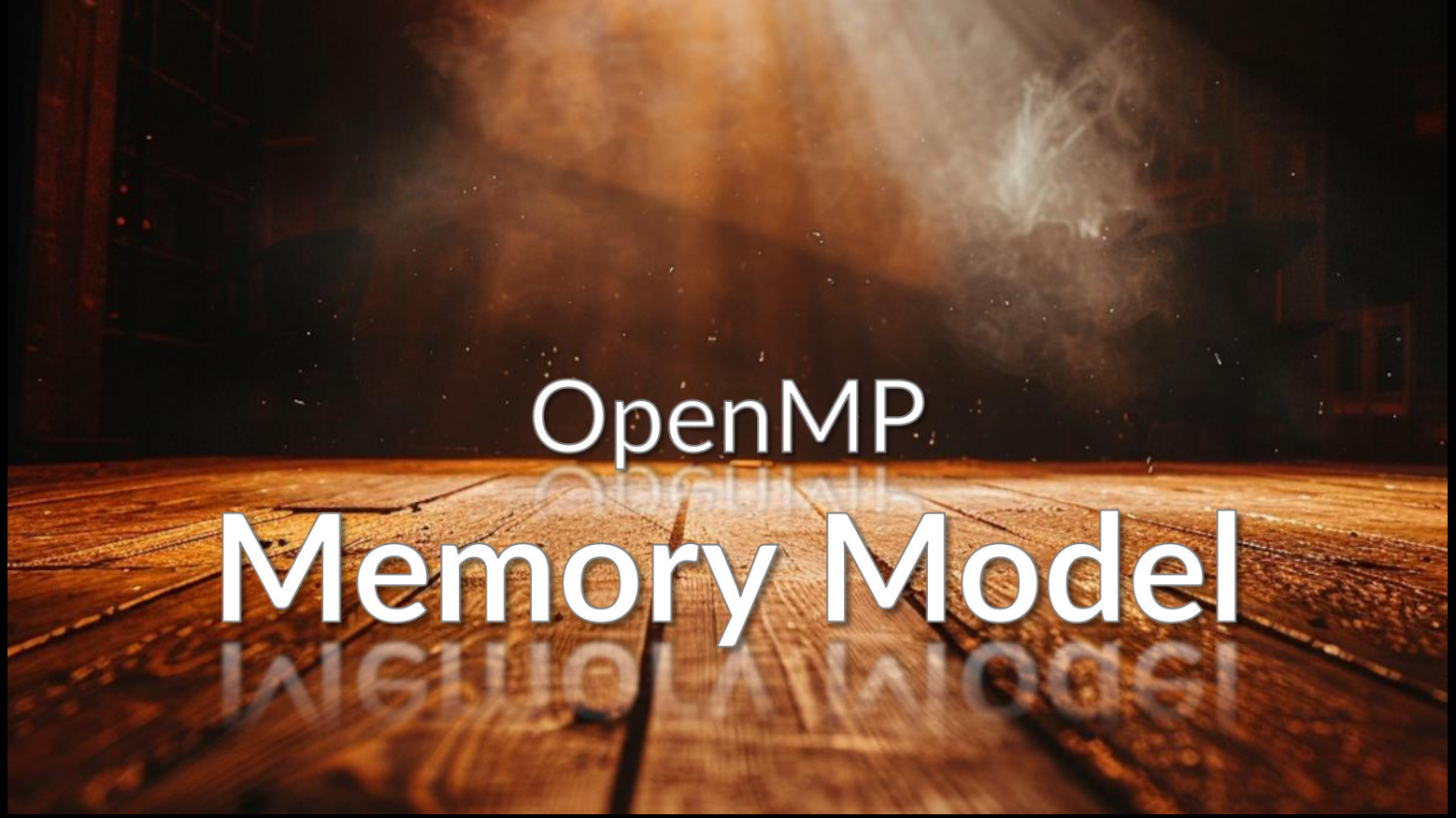


OpenMP
Memory Model

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Physics Department
Ph.D school

Dept. of Math., Inf. and Geosciences

DATA SCIENCE & ARTIFICIAL INTELLIGENCE

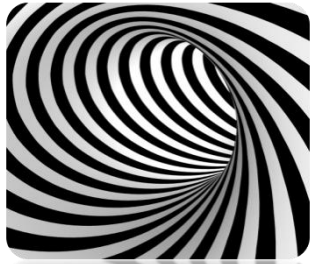
Luca Tornatore
INAF

INAF ISTITUTO NAZIONALE DI ASTROFISICA

OpenMP Outline

Introduction & Concept

Parallel Regions



Parallel Loops

NUMA AWARENESS

MEMORY MODEL

OpenMP Outline

Introduction & Concept

Parallel Regions

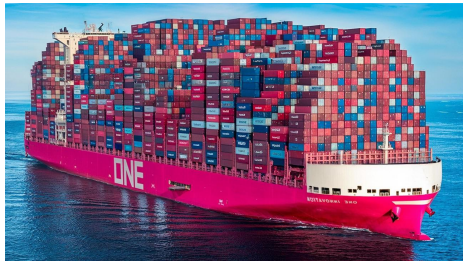
Parallel Loops

NUMA AWARENESS

MEMORY MODEL

OpenMP Outline

Memory Model



Memory Allocators

The OpenMP Memory Model

Why do we need a memory model?

In a shared-memory system, threads see a **common address space.**
As we know, modern CPUs have:

- Private L1/L2 caches per core
- Store buffers that delay writes
- Out-of-order execution that reorders loads/stores

Coherence protocol ensures *eventual* consistency..
"eventual" can mean hundreds or thousands of cycles.

The OpenMP memory model defines:

1. When writes become visible to other threads
2. What ordering guarantees exist at synchronization points
3. How to force visibility with flush operations

You can see the OpenMP memory model as an effective contract between the programmer and the implementation. When a code abides the rules it will behave correctly on any compliant implementation.
If it violates the rules — say, by reading a shared variable without proper synchronization — the behavior is undefined, and the result dependent of whatever the hardware and compiler happen to do.

Temporary view vs Memory

OpenMP introduces a conceptual distinction:

- **Shared memory:** the "ground truth" that all threads eventually agree on.
This is what you'd see if you could freeze the entire system and inspect main memory.

- **Temporary view:** what each thread currently "sees."
This includes values in CPU registers, in store buffers, and in private cache lines that haven't yet been synchronized.

The key insight is that a thread's temporary view may be *stale*. It may contain values that another thread has already overwritten in shared memory, but that update hasn't propagated yet.

```
int x = 0;
int y = 0;
#pragma omp parallel num_threads(2)
 {
    int myid = get_omp_thread_num();
    if ( myid == 0 )
      x = 1;
    else
      y = 1;
    printf("%d sees x= %d and y = %d\n",
           myid, x, y);
 }
```

- Threads's write may still be in their store buffer
- The L1 caches, may even have a stale copy (the update ones in the registers)
- Threads might read 0, or 1, or something else entirely *(in pathological cases on weakly-ordered architectures)*

The FLUSH operations

A **`flush`** is the mechanism that
synchronizes a thread's temporary view with shared memory.
Conceptually, a flush does two things:

1. **Pushes** all pending writes from the thread's temporary view to shared memory
   → makes the calling thread's writes visible to others
2. **Pulls** fresh values from shared memory for subsequent reads
   → makes others' writes visible to the calling thread

Important subtlety: a flush is a point-wise operation.

It synchronizes the calling thread's view with shared memory.
It does not wait for other threads to do anything.
For communication to work, both sides must flush—one to push, one to pull.

Implicit FLUSH points

OpenMP inserts **implicit flush operations** at specific synchronization points.
At these points, a thread's temporary view is synchronized with shared memory.

Implicit flush occurs at:

Construct / Directive	Flush occurs...
#pragma omp barrier	At entry and exit
#pragma omp critical	At entry and exit
#pragma omp parallel	At entry of all threads; at exit (implicit barrier)
#pragma omp for (no nowait)	At exit (implicit barrier)
#pragma omp single (no nowait)	At exit (implicit barrier)
#pragma omp atomic	Only for the memory location being updated
Lock operations	omp_set_lock, omp_unset_lock, etc.

The implicit barrier at the end of parallel, for, sections, and single includes a flush.
This is why, in most simple cases, you don't need explicit flush — synchronization provides it implicitly.

Implicit FLUSH points

```
#pragma omp parallel
{
    #pragma omp for
    for (int i = 0; i < N; i++)
        a[i] = compute(i);

    // implicit barrier + flush here

    #pragma omp for
    for (int i = 0; i < N; i++)
        b[i] = a[i] * 2;   // reads a[]
}
```

In most cases the flushes at implicit barriers are adequate.

However, sync has always a cost and amounts to loose some parallelism.

If you need parallelism more sphosticated than the “bulk” one offered by work-sharing constructs, you need to implement it yourself.

The FLUSH directive

```
#pragma omp flush                    // flush all shared variables
#pragma omp flush(x, y)              // flush only x and y
```

The **flush** directive explicitly forces the thread to synchronize its temporary view with shared memory.

Without arguments: flushes all shared variables visible at that point.
With a list: flushes only the listed variables (more efficient, but be careful!).

`flush` is a point operation: it synchronizes THIS thread's view.
For another thread to see your writes, BOTH threads must flush (or meet at a synchronization point).

When is explicit flush needed?

Most synchronization constructs include implicit flushes, so explicit flush is **rarely needed**.

You might need it for:
- Spin-waiting on a flag variable (busy-wait synchronization)
- Custom lock-free algorithms
- When using atomic with relaxed memory ordering

Spin-Wait with a FLUSH

A classic example where explicit flush is necessary: busy-waiting on a flag.

WRONG — may spin forever

```
int flag = 0;
#pragma omp parallel num_threads(2)
{
   int myid = omp_get_threa_id();
   if ( myid == producer )
     {
       // Producer
       do_work();
       flag = 1;          // write to flag
     }
   else
     {
       // Consumer
       while (flag == 0)  // may read stale!
         ;                // infinite loop?
       use_result();
     }
}
```

CORRECT — with explicit flush

```
int flag = 0;
#pragma omp parallel num_threads(2)
{
   int myid = omp_get_threa_id();
   if ( myid == producer )
     {
       // Producer
       do_work();
       flag = 1;
      #pragma omp flush(flag)  // push
     }
   else
     {
       // Consumer
       while (flag == 0)
         #pragma omp flush(flag) // pull
    use_result();
  }
```

Both threads must flush: the producer flushes to push the write to shared memory; the consumer flushes to pull the fresh value.

*Note: This is a teaching example. In practice, use **atomic** operations (OpenMP 5.0+) for such patterns — they are clearer and have well-defined memory ordering.*

Spin-Wait with a FLUSH

A classic example where explicit flush is necessary: busy-waiting on a flag.

WRONG — may spin forever

```
int flag = 0;
#pragma omp parallel num_threads(2)
{
   int myid = omp_get_threa_id();
   if ( myid == producer )
     {
       // Producer
       do_work();
       flag = 1;          // write to flag
     }
   else
     {
       // Consumer
       while (flag == 0)  // may read stale!
         ;                // infinite loop?
       use_result();
     }
}
```

CORRECT — with explicit flush

```
int flag = 0;
#pragma omp parallel num_threads(2)
{
   int myid = omp_get_threa_id();
   if ( myid == producer )
     {
       // Producer
       do_work();
       flag = 1;
      #pragma omp flush(flag)  // push
     }
   else
     {
       // Consumer
       while (flag == 0)
         #pragma omp flush(flag) // pull
    use_result();
  }
```

Both threads must flush: the producer flushes to push the write to shared memory; the consumer flushes to pull the fresh value.

Note:
*This is a teaching example. In practice, use **atomic** operations (OpenMP 5.0+) for such patterns — they have a clearer well-defined memory ordering.*

Spin-Wait with a FLUSH

```
#pragma omp flush                    // flush all shared variables
#pragma omp flush(x, y)              // flush only x and y
```

The **flush** directive..

a) **It's not a barrier**.
Flush synchronizes the caalling thread's view with shared memory; it doesn't wait for other threads.

b) **Both sides must participate.** For thread A's write to be visible to thread B, thread A must flush (to push) AND thread B must flush (to pull).

c) **Order matters.** In the producer-consumer example, the producer must flush `data` BEFORE setting `flag`, and must flush `flag` after setting it. Otherwise the consumer might see `flag == 1` but read stale `data`.

d) **Compiler reordering is the enemy.** Without flush, the compiler is free to reorder `data = ...` and `flag = 1` if it thinks that's more efficient. Flush acts as a fence that prevents such reordering.

Spin-Wait with a FLUSH

```
#pragma omp flush                    // flush all shared variables
#pragma omp flush(x, y)              // flush only x and y
```

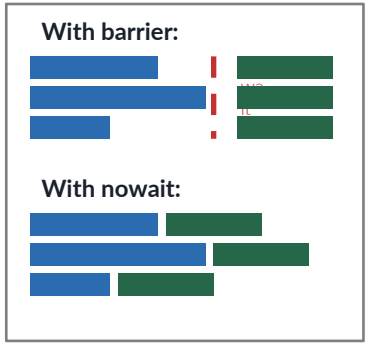
DON'T use **flush** directive when the need is

- that many threads wait for each other; use `barrier`.
- of mutual exclusion; use `critical` or, better, locks.
- to update a shared variable atomically; Use `atomic`.
- that one thread to signal completion to another; use task dependencies (`depend` clause) or barriers (for many-to-many patterns).

The perils of `nowait` clause

By default, worksharing constructs (**for**, **sections**, **single**) have an **implicit barrier** at their end.
All threads must wait until everyone finishes before proceeding.

```
#pragma omp for              // implicit barrier at end
for (int i = 0; i < N; i++)
  a[i] = compute(i);
                             // <-- all threads wait here
#pragma omp for
for (int i = 0; i < N; i++)
  b[i] = other(a[i]);
```



The **nowait** clause removes this barrier: Threads that finish early proceed immediately to the next work.

```
#pragma omp parallel
{
  #pragma omp for nowait
  for (int i = 0; i < N; i++)
    a[i] = compute_a(i);

  #pragma omp for nowait
  for (int i = 0; i < M; i++)
    b[i] = compute_b(i);

}  // implicit barrier here
```

The loops are independent: b[] doesn't depend on a[]. Threads can overlap work.

```
#pragma omp parallel
{
  #pragma omp for nowait
  for (int i = 0; i < N; i++)
    a[i] = compute_a(i);

  #pragma omp for
  for (int i = 0; i < N; i++)
    b[i] = a[i] * 2;    // reads a[]

}
```

The second loop reads a[]. A thread may read a[i] before another thread writes it

A clarification

The word "flush" in OpenMP is historically unfortunate.

It suggests "push dirty data out to memory," which evokes “writing back dirty cache lines to DRAM”.

That is not what flush does, and it's not what atomic release does either.
Neither of them forces dirty cache lines out to DRAM.

A clarification

On a cache-coherent system data does not need to be pushed to DRAM for another thread to see it, because the cache coherence protocol (MESI/MOESI/MESIF) handles that automatically: when thread 1's core requests a line that thread 0's core has dirty, the protocol transfers it cache-to-cache.

What flush and atomic release actually do is enforce ordering :
they constrain when operations become visible to other threads, not where the data physically resides.

The mechanism on hardware is typically:

A compiler barrier: prevent the compiler from reordering memory operations across this point, or from holding values in registers instead of memory.
A hardware fence (sometimes): drain **the store buffer**, or prevent subsequent loads from executing before prior loads complete — depending on the architecture and the fence type.

A clarification

.. drain **the store buffer** ..

The store buffer is the key point.

Stores (i.e. *writes*) don't go directly into the cache; they reside in a per-core store buffer and drain into L1 asynchronously, eventually.

A fence that "flushes" in the OpenMP sense forces the store buffer to drain (or at least forces subsequent operations to wait for it).

Once a store is in L1 and the line is in Modified state, cache coherence takes over — no DRAM write is needed.

A clarification

```
data = (double*)malloc(data_size*sizeof(double));
int flag = 0;

#pragma omp parallel
{
  if ( omp_get_thread_num() == 0)
   {
     for (int i = 0; i < data_size; i++)
       data[i] = ...;
     #pragma omp flush
     flag = 1;
    #pragma omp flush(flag)
   }
  else
   {
     int myflag = flag;
     while (myflag == 0)
      {
        #pragma omp flush(flag)
         myflag = flag;
      }
    #pragma omp flush
     ... //use data
}
```

Ina code like this, the **`flush`**

- Neither **`flush(flag)`** nor **`atomic write release flag = 1`** forces in anyn way the flush of the entire data array.
  DRAM writeback happens lazily, driven by cache eviction policy, and is irrelevant to correctness.
- **`flush(flag)`** or **`atomic write release flag = 1`** ensure that the data[] writes are ordered before the write to flag, such that any thread observing flag == 1 is guaranteed to see the data[] writes

Summary

Concept	What it means	When to use
Temporary view	Each thread may have stale copies of shared variables in registers/cache	Understand why synchronization is needed
Implicit flush	Happens automatically at barrier, critical, parallel entry/exit, lock ops	Most common case — rely on this
Explicit flush	#pragma omp flush — forces sync of thread's view with shared memory	Rare: spin-locks, lock-free code
nowait	Removes implicit barrier from for/sections/single	When next work is independent
barrier	#pragma omp barrier — all threads sync here	When you need ALL threads to finish before proceeding

Advices:

1. Trust implicit synchronization. Barriers, critical sections, and lock operations include flushes. Don't add explicit flushes unless you have a specific reason.

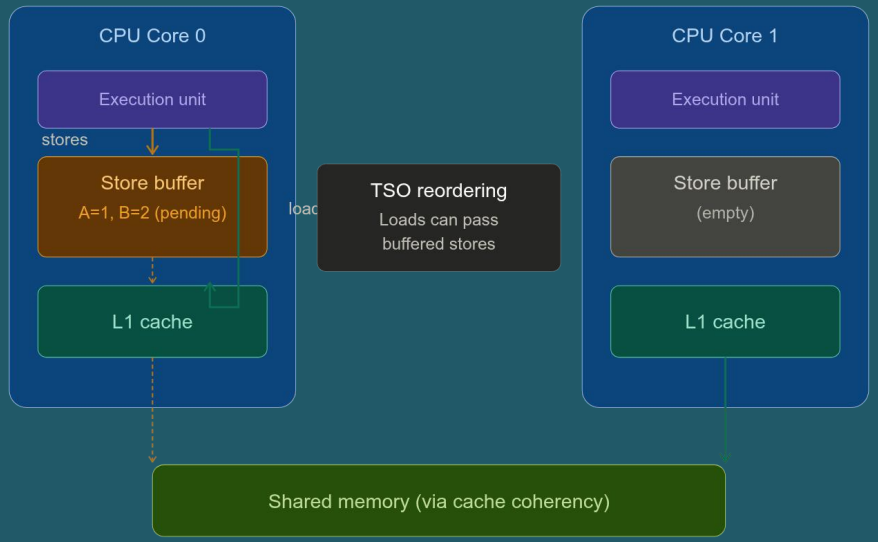
2. If you remove a barrier (nowait), verify independence. If the next computation reads data from the previous, you need the barrier.

3. If you think you need flush, reconsider. Explicit flush is subtle and easy to get wrong. Can you use barrier, critical, or atomic instead?

to be used with examples

Concept **What it means** **When to use**

x86_64 TSO reordering



The store-load reordering

reordering comparison for different architectures

	x86-64 (TSO) Strong	ARM / POWER Weak (relaxed)	
Load → Load	Ordered	Reorderable	— DMB LD
Store → Store	Ordered	Reorderable	— DMB ST
Load → Store	Ordered	Reorderable	— DMB
Store → Load	Reorderable	Reorderable	MFENCE

x86-64 TSO: only store→load can reorder (via store buffer)
ARM/POWER: all orderings require explicit barriers

to be used with examples

Architecture	Model	Store-Store	Load-Load	Store-Load	Load-Store
x86-64	TSO	Ordered	Ordered	**Reorderable**	Ordered
ARM	Relaxed	Reorderable	Reorderable	Reorderable	Reorderable
POWER	Relaxed	Reorderable	Reorderable	Reorderable	Reorderable
RISC-V	RVWMO	Reorderable	Reorderable	Reorderable	Reorderable

The fact that the example code works on x86_64 does not mean that it works everywhere.
Architectures like ARM, POWER and RISC-V were built from scratch with massive parallelism in mind adn have a weaker memory model to increase performance.

to be used with examples

TSO would handle this fine

↓

```
Producer: data = 42; flag = 1;          (stores are ordered)
Consumer: while(flag==0){} use(data);   (if flag==1, data is visible)
```

↑

Compiler hoists this load out of the loop
because it doesn't know about other threads

On x86-64 hardware, if the consumer ever reloads flag from memory and sees 1, TSO guarantees it will also see data == 42.
The store-store ordering ensures the producer's writes are visible in order.
`But the compiler never reloads (with -O3)` — it optimizes the loop loading flag once in a register — so the hardware guarantees are irrelevant.

to be used with examples

withouth flush	with flush
`store flag=1`	`store flag=1`
↓	↓
`[store buffer]`	`[store buffer]`
↓ (drains automatically, 4-10 cycles)	↓ (same drain, but...)
`[L1 cache]`	`MFENCE` CPU stalls here until the buffer is not empty)
↓	↓
`[coherency protocol]`	`[L1 cache]`
↓	↓
`[other cores see it]`	`[coherency protocol]`

The store buffer is not a "holding pen" that requires explicit release.
It drains continuously as part of normal CPU operation.

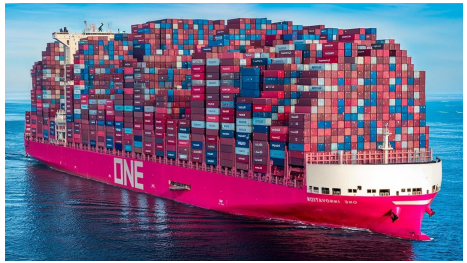
`The flush doesn't *cause* the drain — it just waits for it.`

So in the broken code:

Producer writes flag = 1 → store buffer
Store buffer drains to L1 automatically (~4-10 cycles)
MESI invalidates/updates consumer's cache line (~30-100 cycles)
Consumer's next load from memory would see it.

OpenMP Outline

MEMORY MODEL



MEMORY ALLOCATORS

OpenMP & the Memory

We have seen that the OpenMP memory models specifies when memory changes become visibile toall the threads, and how to enforce that.

We have also mentioned that the strong NUMA hierarchy has a strong impact on the OpenMP performance, because of the different latencies and bandwidths when accessing different DDRx memory banks from a given core.

Let's dig a little bit more into this:

how would you test what is the bandwidth and latencies when accessing from a given core to different memory banks, with the tools that you have until now?

```
memory/
test_bandwidth.c
test_latency.c
```

OpenMP & the Memory

Increasingly more systems have a variety of memory types:
DDR5, HBM/MCDRAM, NVDIMM, CLX, GPU memory,..
with different latency, bandwidth and typical capacity.

How can we explicitly allocate our data on the most adequate memory?

- **proximity**: touch-policy, `numa_alloc_onnode()` when present
- **memory type**: vendor-specific calls, like `hbw_malloc(), cudaMalloc()`, ..., which however force us to write platform-specific code.

OpenMP 5.0 introduced two new concepts:

> **memory spaces** : what kind of storage, in terms of its properties

> **allocators:** how to get the storage, with configurable "traits"

The runtime then maps the request to the hardware, falling back to portable options.

OpenMP's Memory Allocators

default (DDR)	large capac. (NVDIMM)	high bandw. (HBM)	low latency	const (read-only)	type
`omp_default_mem_space`	`omp_large_cap_mem_space`	`omp_high_bw_mem_space`	`omp_low_lat_mem_space`	`omp_const_mem_space`	omp name
`omp_default_mem_alloc`	`omp_large_cap_mem_alloc`	`omp_high_bw_mem_alloc`	`omp_low_lat_mem_alloc`	`omp_const_mem_alloc`	omp allocator

pre-defined memory spaces & allocators

memory space	trait	typical hw
`default`	system default	DDR5
`large capacity`	largest storage capacity	NVDIMM, DDR5
`high bandwidth`	large bandwidth	HBM
`low latency`	low latency	nearest NUMA, GPU scratchpad
`const`	constant values	__constant__ on GPU, read-only pages on CPU

OpenMP's Memory Allocators

default (DDR)	large capac. (NVDIMM)	high bandw. (HBM)	low latency	const (read-only)	type
`omp_default_mem_space`	`omp_large_cap_mem_space`	`omp_high_bw_mem_space`	`omp_low_lat_mem_space`	`omp_const_mem_space`	omp name
`omp_default_mem_alloc`	`omp_large_cap_mem_alloc`	`omp_high_bw_mem_alloc`	`omp_low_lat_mem_alloc`	`omp_const_mem_alloc`	omp allocator

pre-defined memory spaces & allocators

Note:

These are hints, not guarantees. The mapping is implementation-defined.

If the hardware is absent, the runtime silently falls back to `omp_default_mem_space`.

OpenMP's Memory Allocators

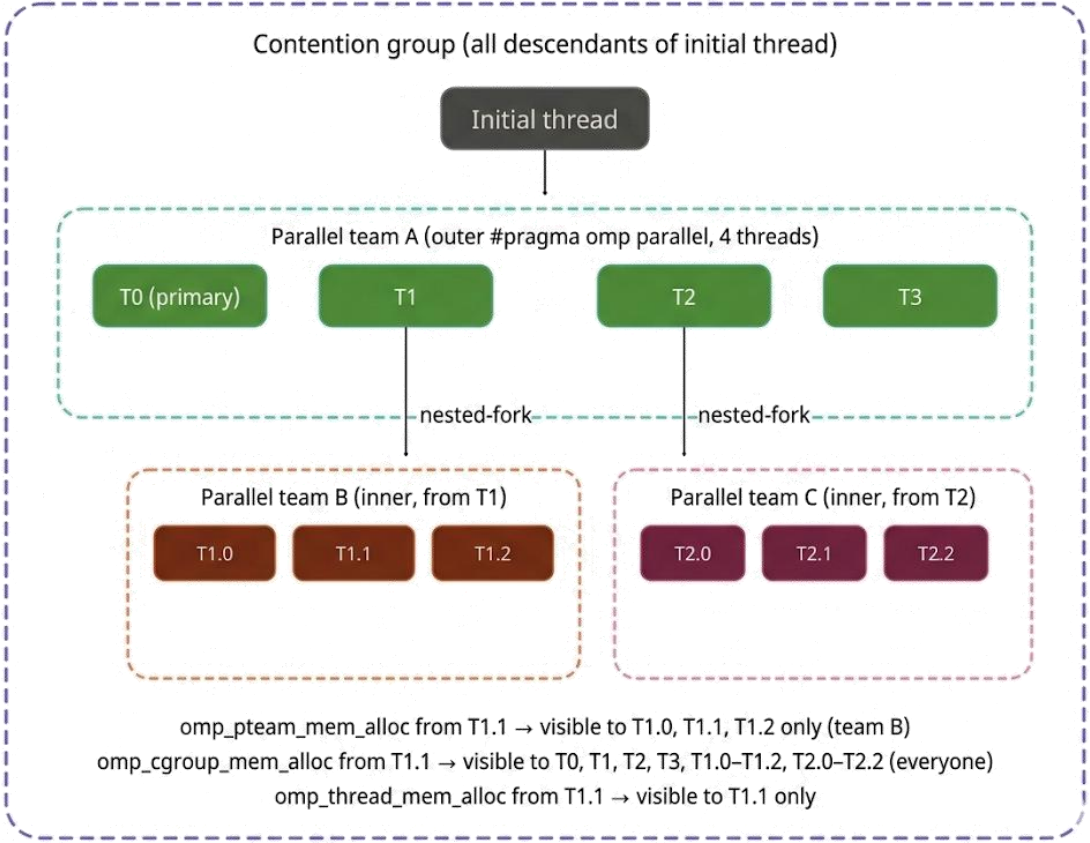
default (DDR)	large capac. (NVDIMM)	high bandw. (HBM)	low latency	const (read-only)	type
`omp_default_mem_space`	`omp_large_cap_mem_space`	`omp_high_bw_mem_space`	`omp_low_lat_mem_space`	`omp_const_mem_space`	omp name
`omp_default_mem_alloc`	`omp_large_cap_mem_alloc`	`omp_high_bw_mem_alloc`	`omp_low_lat_mem_alloc`	`omp_const_mem_alloc`	omp allocator

pre-defined memory spaces & allocators

In addition, there are 3 allocators, whose addressed memory space is implementation-dependent, that specify *who has access to the memory:*

```
omp_cgroup_mem_alloc   access = cgroup   (contention group)
omp_pteam_mem_alloc    access = pteam    (parallel team)
omp_thread_mem_alloc   access = thread
```

OpenMP's access type



The difference btw *contention group* and *parallel* team arises only with nested parallelism.

In brief: the **contention group** is the ensemble of all the threads rooted in the initial thread.

The **parallel team** is the set of threads active in the current (from a thread's point of view) parallel region.

Note: the different access types, cgroup, pteam and thread are mostly useful when dealing with gpu allocations.

The OpenMP allocators traits

trait name	controls	possible values	default
`sync_hint`	contention among threads	`contended, uncontended, serialized, private`	`contended`
`alignment`	byte alignment	any power of 2	1
`access`	visibility scope	`all, cgroup, pteam, thread`	`all`
`poolsize`	max size	any integer	implementation defined
`fallback`	policy on failure	`default_mem_fb, null_fb, abort_fb, allocator_fb`	`default_mem_fb`
`fb_data`	any handle to the allocator		--
`pinned`	paging / migration	`true / false`	`false`
`partition`	NUMA distribution	`environment, nearest, blocked, interleaved`	`environment`

sync_hint is to be considered as a **correctness contract**, not just a performance hint.
If you say serialized and instead you call from multiple threads, you'll get undefined behaviour (for instance, the runtime may skip all locking).

OpenMP allocators in practice

1. API functions — drop-in replacement for malloc/free

```
double *buf = (double*)omp_alloc(N * sizeof(double), omp_high_bw_mem_alloc);
// ... work with buf ...
omp_free(buf, omp_high_bw_mem_alloc);   // NOT just free()!
```

2. allocate directive — controls placement of declared variables

```
double v1[N], v2[N];
#pragma omp allocate(v1, v2) allocator(omp_high_bw_mem_alloc)
// v1, v2 now in HBM (if available). Deallocation automatic at end of scope.
```

3. allocate clause on directives — controls private copies in parallel constructs

```
#pragma omp parallel private(local) allocate(omp_pteam_mem_alloc: local)
{
    // On GPU: local is in shared memory (per-SM scratchpad)
    // On CPU: just a thread-local buffer. Same source code.
}
```

OpenMP allocators in practice

Default fallback (no HBM on machine):

```
double *buf = omp_alloc(
    N * sizeof(double),
    omp_high_bw_mem_alloc);

// 1. Try high_bw_mem_space → fail
// 2. fallback = default_mem_fb
//    → retry in default → DDR → OK
// Code doesn't crash.
// Just uses slower memory.
```

Building an explicit chain:

```
omp_alloctrait_t traits[2];
traits[0].key   = omp_atk_fallback;
traits[0].value = omp_atv_allocator_fb;

traits[1].key   = omp_atk_fb_data;
traits[1].value = (omp_uintptr_t)
            omp_large_cap_mem_alloc;

tiered = omp_init_allocator(omp_high_bw_mem_space, 2, traits);
// HBM → large_cap → default → NULL
```

Quick experiment without recompiling:

```
export OMP_ALLOCATOR=omp_high_bw_mem_alloc

export OMP_ALLOCATOR="omp_low_lat_mem_space:pinned=true,partition=nearest"  # <--- GCC extension
```

OpenMP Mem Spaces recap

Desired trait	OpenMP's way	Example use case
Allocate in HBM	`omp_alloc(sz, omp_high_bw_mem_alloc)`	Bandwidth-bound kernels
Aligned (SIMD)	`omp_atk_alignment = 64`	AVX-512 inner loops
Pinned memory	`omp_atk_pinned = true`	RDMA, GPU DMA
GPU shared mem	`allocate(omp_pteam_mem_alloc: v)`	target teams scratchpad
NUMA-local	`partition = nearest`	Per-thread arrays
Portable fallback	`fallback = allocator_fb chain`	HBM + non-HBM machines
Quick experiment	`export OMP_ALLOCATOR=...`	No recompilation

Summary

Main topics missed in this very short course

1. locks
2. synchronization using `atomic` clauses
3. task-parallelism

that’s all, have fun